

# Refactoring

Improving the design of existing code — safely

## Course block

Definition and motivation

Code smells and diagnosis

Core refactorings

Tool-assisted and AI-assisted  
refactoring

Full modern Java example

Pedro García López · TAP

# Learning goals

[OVERVIEW](#)

By the end, students should be able to refactor without changing behavior.

## Recognize smells

Long Method, Duplicate Code, Feature Envy, Primitive Obsession, Switch Statements...

## Apply small refactorings

Extract Method, Move Method, Rename, Replace Temp with Query, Extract Class...

## Refactor safely

Characterization tests, small commits, IDE refactorings, continuous feedback.

## Use AI responsibly

Ask for plans, preserve tests, review diffs, enforce architecture.

# What is refactoring?

CONCEPT

A series of small behavior-preserving transformations.

## Definition

Refactoring changes the internal structure of software so that it is easier to understand and cheaper to modify, without changing observable behavior.

**Behavior must remain stable.**

```
// Same input
invoice.generateReceipt(order)

// Same observable output
"Total: 42.50 EUR"
```

Refactoring is not rewriting from scratch. It is controlled change: small step, compile, test, repeat.

# Why refactor?

MOTIVATION

Because software spends most of its life being changed.

## Understand

Make code readable for humans and future maintainers.

## Change

Prepare the design for the next feature.

## Test

Isolate logic so it can be verified cheaply.

## Review

Expose intention so humans and AI can modify safely.

```
// Bad motivation for refactoring
"I want to make it prettier"

// Better motivation
"This feature requires changes in 7 places;
let's reduce the coupling first."
```

# The two hats

DISCIPLINE

Do not mix adding behavior and restructuring behavior.

## Adding functionality

- Add a new capability
- Add or change tests
- Expect behavior to change
- Implement until tests pass

## Refactoring

- No new feature
- Existing tests remain green
- Internal design changes
- Small reversible steps

**Rule: swap hats often, but wear only one at a time.**

# Safety net: tests first

DISCIPLINE

Without tests, refactoring becomes guessing.

```
@Test
void textReceiptKeepsExistingBehavior() {
    Order order = sampleOrder();

    String receipt = new ReceiptService().text(order);

    assertThat(receipt).contains("Amount owed: 42.50");
    assertThat(receipt).contains("Reward points: 3");
}
```

## Refactoring test pyramid

Characterization tests for legacy behavior

Unit tests for extracted logic

Contract tests for strategies/interfaces

Integration tests for end-to-end output

**Green → small change → green again**

# A practical refactoring loop

WORKFLOW

Make progress through tiny, verified transformations.



```
git commit -m "refactor: extract receipt line formatting"

// Good refactoring commits are small and reversible.
// If tests fail, you know where the problem is.
```

# Code smells

SMELLS

A smell is a symptom, not an automatic error.

## Bloaters

Long Method · Large Class · Long Parameter List · Primitive Obsession

## Change preventers

Divergent Change · Shotgun Surgery · Parallel Inheritance

## OO abusers

Switch Statements · Refused Bequest · Temporary Field

## Dispensables

Duplicate Code · Dead Code · Lazy Class · Comments as deodorant

## Couplers

Feature Envy · Inappropriate Intimacy · Message Chains

## Testability smells

Hidden dependencies · Static calls · Real services in tests

# Smell family: Bloaters

SMELLS

Code grows until intention disappears.

```
void process(Order o) {  
    // validate input  
    // compute discounts  
    // calculate taxes  
    // generate text  
    // send email  
    // update database  
    // notify analytics  
}
```

## Common bloaters

Long Method: too many responsibilities

Large Class: too much knowledge

Long Parameter List: missing object

Primitive Obsession: domain concepts represented as String/int

**Typical refactorings: Extract Method · Extract Class · Introduce Parameter Object · Replace Primitive with Object**

# Smell family: Change preventers

SMELLS

Small features require edits everywhere.

```
// Adding JSON export requires editing:  
ReportController.java  
ReportService.java  
EmailService.java  
InvoicePrinter.java  
CliParser.java  
Tests...  
  
// Symptom: the design has no extension  
point.
```

## Typical smells

Divergent Change: one class changes for many unrelated reasons

Shotgun Surgery: one change touches many classes

Parallel Inheritance Hierarchies: every subclass family mirrors another

**Typical refactorings: Extract Class · Move Method · Introduce Strategy · Replace Conditional with Polymorphism**

# Smell family: Object-orientation abusers

SMELLS

The code uses classes but not polymorphism.

```
double price(String type, int days) {  
    return switch (type) {  
        case "REGULAR" -> 2 + Math.max(0, days - 2) * 1.5;  
        case "PREMIUM" -> days * 4;  
        case "FREE" -> 0;  
        default -> throw new IllegalArgumentException(type);  
    };  
}
```

## Typical smells

- Switch Statements controlling behavior
- Temporary Field used only sometimes
- Refused Bequest from poor inheritance
- Alternative Classes with Different Interfaces

**Typical refactorings: Replace Conditional with Polymorphism · Introduce Strategy · Pull Up Method · Adapter**

# Smell family: Couplers

SMELLS

One class knows too much about another.

```
// Feature Envy: pricing logic lives outside
OrderLine
double lineTotal(OrderLine line) {
    return line.getProduct().getBasePrice()
        * line.getQuantity()
        *
    line.getProduct().getTaxCategory().rate();
}
```

```
// Message Chain: fragile navigation
String city = order.customer()
    .address()
    .country()
    .capital()
    .name();
```

## Typical refactorings

Move Method: put behavior near the data

Hide Delegate: reduce navigation chains

Extract Class: split intimate collaborators

Introduce Facade: protect clients from subsystem details

# Smell spotting: ask better questions

SMELLS

A smell is a prompt for investigation.

## Where does change hurt?

How many files change for one feature?

## Who owns this logic?

Does the method use another object more than itself?

## What is duplicated?

Same condition? Same loop? Same formatting?

## Can I name this block?

If yes, Extract Method may reveal intention.

## Can I test this alone?

If no, dependencies may be hidden or too concrete.

## Is the type system helping?

Domain concepts represented as String/int?

# Refactoring catalog: useful moves

CATALOG

Names help teams discuss safe transformations.

## Composing methods

Extract Method · Inline Method · Replace Temp with Query

## Moving features

Move Method · Move Field · Extract Class · Inline Class

## Organizing data

Encapsulate Collection · Replace Primitive with Object · Introduce Parameter Object

## Simplifying conditionals

Decompose Conditional · Replace Conditional with Polymorphism · Guard Clauses

## Simplifying calls

Rename Method · Separate Query from Modifier · Preserve Whole Object

## Inheritance/polymorphism

Pull Up Method · Push Down Method · Extract Superclass · Strategy

# Refactoring: Extract Method

[CATALOG](#)

Turn a block of code into a named operation.

```
// Before
void printReceipt(Order o) {
    System.out.println("Customer: " + o.customer().name());
    double total = 0;
    for (OrderLine l : o.lines()) {
        total += l.price() * l.quantity();
    }
    System.out.println("Total: " + total);
}
```

```
// After
void printReceipt(Order o) {
    printHeader(o);
    System.out.println("Total: " + total(o));
}

private void printHeader(Order o) {
    System.out.println("Customer: " + o.customer().name());
}

private double total(Order o) { ... }
```

**Use when a code fragment can be named. Good names are the main benefit, not shorter code.**

# Refactoring: Move Method

[CATALOG](#)

Move behavior to the class that owns the data.

```
// Before: service envies OrderLine
class ReceiptService {
    double lineTotal(OrderLine line) {
        return line.unitPrice() * line.quantity();
    }
}
```

```
// After: behavior near data
record OrderLine(String product, int quantity, Money unitPrice)
{
    Money total() {
        return unitPrice.multiply(quantity);
    }
}

class ReceiptService {
    Money total(Order o) {
        return o.lines().stream()
            .map(OrderLine::total)
            .reduce(Money.ZERO, Money::add);
    }
}
```

**Typical trigger: Feature Envy. The method calls many getters on another object.**

# Refactoring: Replace Temp with Query

[CATALOG](#)

Replace temporary variables with intention-revealing methods.

```
// Before
double basePrice = quantity * itemPrice;
if (basePrice > 1000) {
    return basePrice * 0.95;
}
return basePrice * 0.98;
```

```
// After
if (basePrice() > 1000) {
    return basePrice() * 0.95;
}
return basePrice() * 0.98;

double basePrice() {
    return quantity * itemPrice;
}
```

Why? It makes future extraction easier and removes local-variable noise. Do not use it when recomputation is expensive without measuring.

# Refactoring: Replace Conditional with Polymorphism

Move each case into its own implementation.

[CATALOG](#)

```
// Before
switch (plan) {
    case FREE -> 0;
    case PRO -> users * 12;
    case ENTERPRISE -> base + users * 8;
}
```

```
// After
interface PricingPlan {
    Money monthlyPrice(Account account);
}

class FreePlan implements PricingPlan { ... }
class ProPlan implements PricingPlan { ... }
class EnterprisePlan implements PricingPlan { ... }
```

**Use when a conditional selects behavior according to a type code and the behavior is likely to grow.**

# Refactoring: Introduce Parameter Object

[CATALOG](#)

Group parameters that travel together.

```
// Before
List<Order> findOrders(LocalDate from,
                      LocalDate to,
                      String status,
                      String customerId) { ... }
```

```
// After
record OrderSearch(LocalDate from,
                  LocalDate to,
                  String status,
                  String customerId) {}

List<Order> findOrders(OrderSearch query) { ... }
```

**Modern Java records are excellent parameter objects: small, immutable, readable.**

# Refactoring: Replace Primitive with Object

[CATALOG](#)

Give domain concepts their own type.

```
// Before
record Student(String name, double grade) {}

boolean passed(Student s) {
    return s.grade() >= 5.0;
}
```

```
// After
record Grade(double value) {
    Grade {
        if (value < 0 || value > 10) throw new
        IllegalArgumentException();
    }
    boolean isPass() { return value >= 5.0; }
}
record Student(String name, Grade grade) {}
```

**Benefit: validation and behavior move into the type system.**

# Refactoring: Encapsulate Collection

CATALOG

Protect invariants and avoid leaking mutable internals.

```
// Before
class Order {
    public List<OrderLine> lines = new ArrayList<>();
}

order.lines.add(invalidLine);
```

```
// After
class Order {
    private final List<OrderLine> lines = new ArrayList<>();

    public void addLine(OrderLine line) {
        requireValid(line);
        lines.add(line);
    }

    public List<OrderLine> lines() {
        return List.copyOf(lines);
    }
}
```

**Expose behavior, not mutable representation.**

# Tool-assisted and AI-assisted refactoring

MODERN

Tools can help, but they do not replace design judgment.

## IDE refactorings

- Rename safely across project
- Extract Method/Class/Interface
- Inline method/variable
- Change signature
- Move members

## AI refactoring workflow

- Ask for a plan before edits
- Require tests to remain unchanged
- Review diff, not just final code
- Preserve public API unless specified
- Keep commits small

```
Prompt: "Refactor only. Do not change external behavior.  
Run/adjust tests and explain each refactoring applied."
```

# Modern running example

EXAMPLE

A messy receipt generator in a student project.

## Scenario

Input: an order with lines and a customer

Output: text receipt today

New requirements: HTML and JSON receipts

Pricing rules will change next month

Students must preserve existing output

## Initial smells

Long Method in ReceiptService

Switch on customer type

Feature Envy on OrderLine

Duplicate formatting logic coming soon

Primitive Obsession for money and type codes

# Dirty code: too many reasons to change

EXAMPLE

One method mixes pricing, formatting, totals and loyalty points.

```
class ReceiptService {
    String receipt(Order order) {
        double total = 0;
        int points = 0;
        String result = "Receipt for " + order.customer().name() + "\n";

        for (OrderLine line : order.lines()) {
            double amount = line.unitPrice() * line.quantity();
            switch (order.customer().type()) {
                case "VIP" -> amount *= 0.90;
                case "STUDENT" -> amount *= 0.95;
            }
            points += amount > 50 ? 2 : 1;
            result += line.productName() + " " + amount + "\n";
            total += amount;
        }

        result += "Amount owed: " + total + "\n";
        result += "Reward points: " + points;
        return result;
    }
}
```

# Before refactoring: lock behavior with tests

EXAMPLE

Characterization tests describe what the current code does.

```
@Test
void existingTextReceiptIsPreserved() {
    Order order = Fixtures.vipOrder();

    String receipt = new ReceiptService().receipt(order);

    assertThat(receipt).isEqualTo("""
        Receipt for Ada
        Book 27.0
        Pen 4.5
        Amount owed: 31.5
        Reward points: 2""").trim());
}
```

## Why exact-output tests?

They protect observable behavior

They make small refactorings safe

They reveal accidental formatting changes

Later, add lower-level tests for extracted logic

# Step 1: Extract Method

EXAMPLE

Name the pricing and line-formatting intentions.

```
String receipt(Order order) {  
    double total = 0;  
    int points = 0;  
    String result = header(order);  
  
    for (OrderLine line : order.lines()) {  
        double amount = amountFor(line, order.customer());  
        points += rewardPointsFor(amount);  
        result += lineFor(line, amount);  
        total += amount;  
    }  
    return result + footer(total, points);  
}
```

## What improved?

The loop now reads like a story

Each helper can be tested separately

Formatting and pricing are named

No behavior change yet

**Refactoring applied: Extract Method + Rename Variable**

## Step 2: Replace temp with query

EXAMPLE

Make totals reusable for text, HTML and JSON outputs.

```
// Before
double total = 0;
for (OrderLine line : order.lines()) {
    double amount = amountFor(line, customer);
    total += amount;
}
return footer(total, points);
```

```
// After
Money totalAmount(Order order) {
    return order.lines().stream()
        .map(line -> amountFor(line, order.customer()))
        .reduce(Money.ZERO, Money::add);
}

int totalPoints(Order order) { ... }
```

**The text output can now share totals with future HTML/JSON formatters.**

## Step 3: Move Method

EXAMPLE

Pricing of a line belongs close to the line data.

```
// Before
double amountFor(OrderLine line, Customer customer) {
    double amount = line.unitPrice() * line.quantity();
    return discount(customer.type(), amount);
}
```

```
// After
record OrderLine(String productName, int quantity, Money
unitPrice) {
    Money subtotal() {
        return unitPrice.multiply(quantity);
    }
}

Money amountFor(OrderLine line, Customer customer) {
    return discount(customer.type(), line.subtotal());
}
```

Refactoring applied: Move Method. The service no longer recomputes what an OrderLine already knows.

## Step 4: Replace conditional with Strategy

EXAMPLE

Customer-specific pricing becomes extensible.

```
// Before
switch (customer.type()) {
  case "VIP" -> amount *= 0.90;
  case "STUDENT" -> amount *= 0.95;
}
```

```
interface DiscountPolicy {
  Money apply(Money subtotal);
}

class VipDiscount implements DiscountPolicy {
  public Money apply(Money subtotal) {
    return subtotal.multiply(0.90);
  }
}

class StudentDiscount implements DiscountPolicy { ... }
```

**Now adding TEACHER discount means adding a new class, not editing a central switch.**

# Step 5: Extract formatter strategy

EXAMPLE

Support text, HTML and JSON without duplicating totals.

```
interface ReceiptFormatter {  
    String format(ReceiptView view);  
}  
  
class TextReceiptFormatter implements ReceiptFormatter { ... }  
class HtmlReceiptFormatter implements ReceiptFormatter { ... }  
class JsonReceiptFormatter implements ReceiptFormatter { ... }  
  
record ReceiptView(String customer, List<ReceiptLine> lines,  
                   Money total, int points) {}
```

## Refactorings combined

Extract Class: ReceiptView

Introduce Strategy: ReceiptFormatter

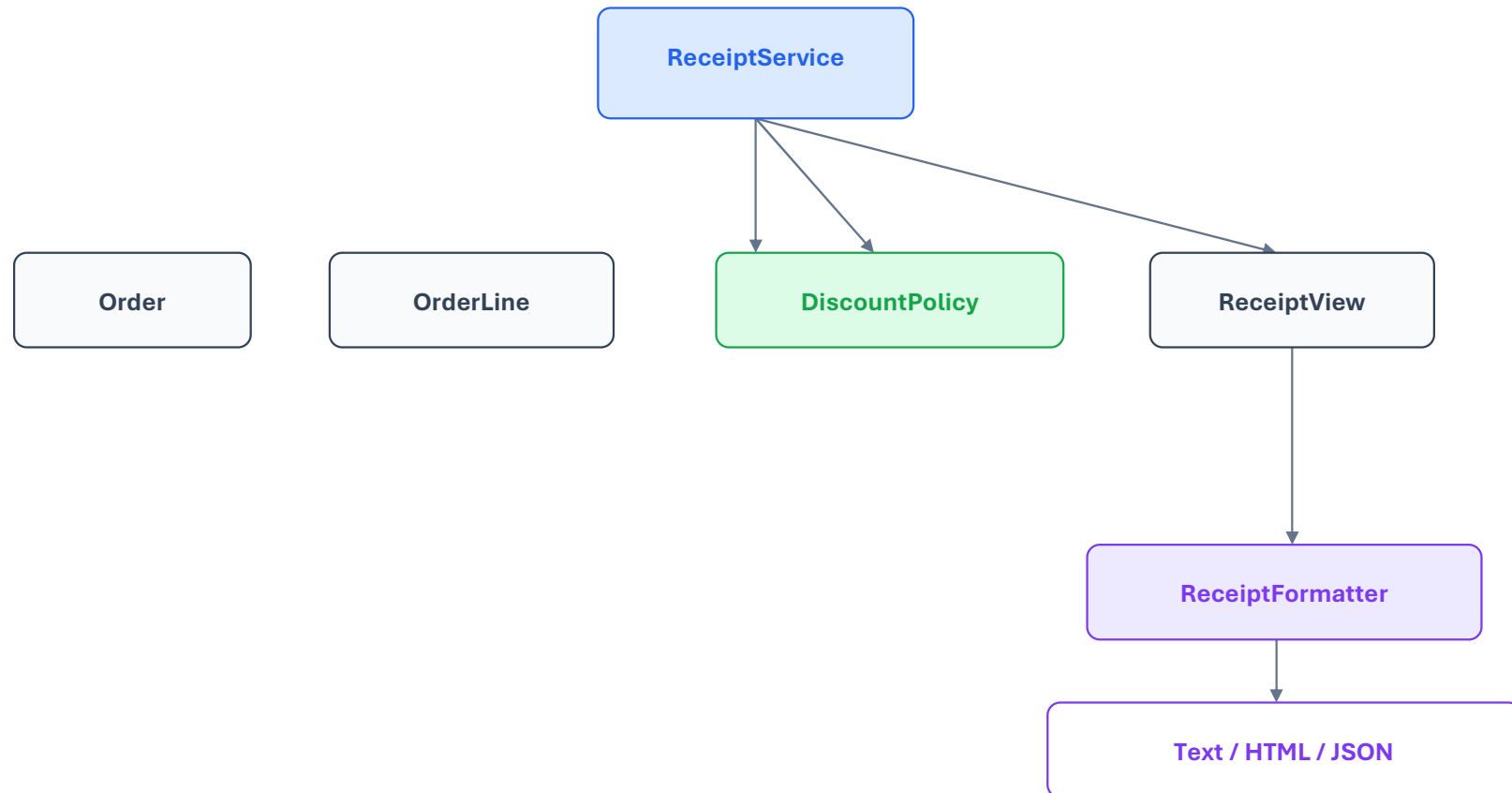
Move Method: line subtotal

Replace Conditional with Polymorphism:  
discount policies

# Resulting design

EXAMPLE

Responsibilities are smaller and extension points are explicit.



**After refactoring, adding a new formatter or discount policy is local, explicit and testable.**

# Clean code after refactoring

EXAMPLE

The public method now orchestrates collaborators.

```
class ReceiptService {
    private final DiscountPolicy discount;
    private final ReceiptFormatter formatter;

    String receipt(Order order) {
        List<ReceiptLine> lines = order.lines().stream()
            .map(this::toReceiptLine)
            .toList();

        ReceiptView view = new ReceiptView(
            order.customer().name(),
            lines,
            total(lines),
            rewardPoints(lines)
        );
        return formatter.format(view);
    }

    private ReceiptLine toReceiptLine(OrderLine line) {
        return new ReceiptLine(line.productName(),
            discount.apply(line.subtotal()));
    }
}
```

# What changed and what did not?

EXAMPLE

The refactoring succeeds only if behavior is preserved.

## Did not change

- Same public receipt behavior
- Same totals and reward points
- Same current text output
- Same input objects

## Improved

- Pricing isolated
- Formatting isolated
- Shorter methods
- More testable components
- New formats and policies are local changes

# Spec-driven refactoring assignment

ASSIGNMENT

Students should document intent before editing code.

```
# proposal.md
```

```
## Problem
```

```
ReceiptService mixes pricing, formatting and reward points.  
Adding HTML output will duplicate logic.
```

```
## Refactoring goal
```

```
Keep the current text receipt unchanged while extracting:
```

- DiscountPolicy strategy
- ReceiptFormatter strategy
- ReceiptView data object

```
## Non-goals
```

```
Do not change public output.
```

```
Do not add new receipt formats until tests are green.
```

```
# tasks.md
```

- [ ] Add characterization test for current receipt
- [ ] Extract line formatting method
- [ ] Move subtotal to OrderLine
- [ ] Introduce DiscountPolicy
- [ ] Introduce ReceiptView
- [ ] Introduce ReceiptFormatter
- [ ] Verify output is unchanged
- [ ] Only then add HtmlReceiptFormatter

# Refactoring checklist

PRACTICE

Use this before submitting or reviewing.

---

Tests existed before the first refactoring step

Each commit contains one conceptual change

Public behavior is unchanged unless explicitly requested

Names now reveal intention better than before

Responsibilities moved closer to the data they use

The next expected feature is easier to add

AI-generated changes were reviewed, not blindly accepted

# When not to refactor

PRACTICE

Refactoring is a professional tool, not a ritual.

## Be careful when...

There are no tests and behavior is unclear

A deadline requires a tiny safe fix

The code is about to be deleted

You are changing public APIs used by others

Performance is critical and unmeasured

## Do refactor when...

You are about to add a feature

The same change appears in several places

Tests can protect behavior

A code review reveals design confusion

AI is producing messy patches

# Summary

WRAP-UP

Refactoring is how design improves after code exists.

Smell → Test → Small refactoring → Green tests → Commit → Repeat

## Key idea

Change structure, not behavior.

## Key discipline

Use tests and small steps.

## Key design move

Move responsibility to the right place.

## Key modern angle

Use IDEs and AI, but keep human review and specs.

# References and source material

REFERENCES

Classic ideas, updated with modern Java practice.

---

Martin Fowler, Refactoring: Improving the Design of Existing Code.

Kent Beck, Extreme Programming Explained.

Steve McConnell, Code Complete.

Original TAP refactoring example: video rental statement and Fowler refactorings.

Course practice: apply refactorings using tests, IDE support and spec-driven changes.

**Suggested student exercise: choose one dirty method, write characterization tests, apply three refactorings, and explain why behavior did not change.**